

Because every class, function, and method should have a docstring, you might be tempted to write only the minimal amount of documentation and move on. Without a `TODO` comment, it's easy to forget that this docstring will eventually need rewriting.

PEP 257 contains further documentation on docstrings at <https://www.python.org/dev/peps/pep-0257/>.

Type Hints

Many programming languages have *static typing*, meaning that programmers must declare the data types of all variables, parameters, and return values in the source code. This allows the interpreter or compiler to check that the code uses all objects correctly before the program runs. Python has *dynamic typing*: variables, parameters, and return values can be of any data type or even change data types while the program runs. Dynamic languages are often easier to program with because they require less formal specification, but they lack the bug-preventing advantages that static languages have. If you write a line of Python code, such as `round('forty two')`, you might not realize that you're passing a string to a function that accepts only `int` or `float` arguments until you run the code and it causes an error. A statically typed language gives you an early warning when you assign a value or pass an argument of the wrong type.

Python's *type hints* offer optional static typing. In the following example, the type hints are in bold:

```
def describeNumber(number: int) -> str:
    if number % 2 == 1:
        return 'An odd number. '
    elif number == 42:
        return 'The answer. '
    else:
        return 'Yes, that is a number. '

myLuckyNumber: int = 42
print(describeNumber(myLuckyNumber))
```
